Position-Resolved Likelihood Inference for Atom Gradiometer Image Data

Contents

1 Setup and Notation

1.1 Gradiometer Architecture

We consider an atom gradiometer composed of two atom interferometers (AIs) operated simultaneously, one with atoms sourced at height $z = 0\,\mathrm{m}$ and one at $z = 100\,\mathrm{m}$, indexed hereafter by $z \in \mathcal{Z} = \{0,\, 100\}$. Each AI independently measures the accumulated phase along its atom trajectories. Common-mode laser-phase noise cancels in the differential signal, while a spatially-dependent differential phase (e.g. from a gravitational wave or dark-matter field) is retained.

A run consists of N_{shots} shots indexed $i = 1, \ldots, N_{\text{shots}}$, acquired at times t_i. Each shot produces two 2D fluorescence images per AI: one for ground-state atoms ($s = g$, bright port) and one for excited-state atoms ($s = e$, dark port). Images share a common pixel grid with bin centres $\mathbf{x}_b = (x_b, y_b)$, bin area $\Delta A = \Delta x\, \Delta y$, and N_b bins total.

1.2 Parameters

Science signal. The target quantity is the differential phase signal injected into AI $z = 100$:

$$\delta\phi_i(\beta) = A_s \sin(2\pi f\, t_i) + A_c \cos(2\pi f\, t_i), \qquad \beta = (A_s,\, A_c), \tag{1}$$

where the signal frequency f is assumed known a priori, leaving two free parameters. The signal amplitude and phase are $A = \sqrt{A_s^2 + A_c^2}$ and $\varphi = \arctan 2(A_c, A_s)$.

Common laser phase. The common laser phase $\phi_i \sim \mathrm{Uniform}(0, 2\pi)$ is shared by both AIs within each shot, drawn independently across shots, and unobserved. It is a latent variable; how we handle it in the likelihood is discussed in Section **??**.

Effective AI phase. The total phase entering the transition probability for AI z in shot i is

$$\Phi_{iz}(\phi_i, \beta) = \phi_i + \psi_z\big(\delta\phi_i(\beta)\big), \qquad \psi_0 = 0, \quad \psi_{100} = \delta\phi_i(\beta). \tag{2}$$

Cloud nuisance parameters. The initial atomic cloud for AI z in shot i is characterised by a Gaussian phase-space distribution with parameters

$$\eta_{iz} = \big(\mu_{x_0},\, \mu_{y_0},\, \mu_{v_x},\, \mu_{v_y},\, \sigma_x,\, \sigma_y,\, \sigma_{v_x},\, \sigma_{v_y}\big)_{iz}, \tag{3}$$

the transverse centre-of-mass (COM) positions, COM velocities, and Gaussian widths. These vary shot-to-shot and across AIs. Like ϕ_i, they are unobserved and must be handled carefully in the likelihood; see Section **??**.

2 Semiclassical Phase-Space Map (PSMAP)

2.1 PSMAP Surrogate

The semiclassical simulation (ais++) integrates the interferometer sequence for a regular 4D grid of initial transverse phase-space coordinates $\mathbf{q}_0 = (x_0, y_0, v_{x0}, v_{y0})$ at fixed source height and longitudinal velocity. For each grid point and each output port p it stores: (i) wavepacket path amplitudes $a_{0,p}(\mathbf{q}_0)$ and $a_{1,p}(\mathbf{q}_0)$; (ii) accumulated phase difference $\Delta\phi_p(\mathbf{q}_0)$; (iii) interference flag $\iota_p \in \{0, 1\}$; and (iv) output state label $s_p \in \{g, e\}$. At runtime, values are obtained by quadrilinear interpolation on the grid. A separate PSMAP is computed for each AI.

2.2 Transition Probabilities

The probability that an atom with initial condition $\mathbf{q}_0$ is detected at port p given total AI phase Φ is

$$P_p(\mathbf{q}_0;\,\Phi) = a_{0,p}^2 + a_{1,p}^2 + \iota_p \cdot 2\, a_{0,p}\, a_{1,p}\, \cos\!\big(\Delta\phi_p(\mathbf{q}_0) + \Phi\big). \tag{4}$$

The per-state probabilities are obtained by summing over ports with matching state label:

$$p_s(\mathbf{q}_0;\,\Phi) = \sum_{p:\, s_p = s} P_p(\mathbf{q}_0;\,\Phi), \qquad s \in \{g, e\}. \tag{5}$$

Because the simulation retains only the two dominant interferometry paths,

$$p_g(\mathbf{q}_0;\,\Phi) + p_e(\mathbf{q}_0;\,\Phi) \le 1; \tag{6}$$

atoms not accounted for are lost to pruned paths. In particular, $p_e \neq 1 - p_g$ in general; both are computed independently.

2.3 Ballistic Trajectory Map

Under free-flight propagation over detection time T_{det}, the final transverse position is

$$\mathbf{x}^f(\mathbf{q}_0) = \begin{pmatrix} x_0 + T_{\text{det}}\, v_{x0} \\ y_0 + T_{\text{det}}\, v_{y0} \end{pmatrix}. \tag{7}$$

Both output states follow the same trajectory (no state-dependent force before imaging). With default parameters ($T_{\text{det}} = 3.8\,\text{s}$, $\sigma_x = 100\,\mu\text{m}$, $\sigma_{v_x} = 309\,\mu\text{m/s}$), the cloud expands to $\sigma_{x^f} \approx 1.2\,\text{mm}$ at detection.

3 Generative Model for Pixel Counts

3.1 Expected Pixel Counts

The expected count of state-s atoms in pixel b for AI z, shot i, conditioned on ϕ_i and η_{iz}, is

$$\lambda_{izsb}(\phi_i, \eta_{iz}) = \Lambda_{iz} \int d\mathbf{q}_0\; p_0(\mathbf{q}_0 \mid \eta_{iz})\; p_s\big(\mathbf{q}_0;\, \Phi_{iz}(\phi_i, \beta)\big)\; \mathbf{1}\big[\mathbf{x}^f(\mathbf{q}_0) \in \text{bin } b\big], \tag{8}$$

where Λ_{iz} is the launched atom count (fixed from observations, see Section **??**) and the indicator function $\mathbf{1}[\mathbf{x}^f \in \text{bin } b]$ selects atoms whose final position falls in pixel b.

3.2 Small-Pixel Approximation

Both p_0 and p_s vary on the scale of the cloud ($\sigma_{x^f} \sim 1\,\text{mm}$), while pixels are $\Delta x \sim 6\,\mu\text{m}$ — a ratio of ~ 170. The integrand in (**??**) is therefore constant within each bin, and we evaluate it at the bin centre $\mathbf{x}_b$:

$$\lambda_{izsb}(\phi_i, \eta_{iz}) \approx \Lambda_{iz}\, \Delta A \quad \underbrace{f_{\text{cloud}}(\mathbf{x}_b;\, \eta_{iz})}_{\text{cloud density at pixel centre}} \quad \times\; \underbrace{\mathbb{E}_{(v_{x0}, v_{y0}) \sim p(\cdot \mid \mathbf{x}_b, \eta_{iz})}\big[p_s(\mathbf{q}_0(\mathbf{x}_b, \mathbf{v}_0);\, \Phi_{iz})\big]}_{\text{velocity-averaged transition prob.}}, \tag{9}$$

where $\mathbf{q}_0(\mathbf{x}_b, \mathbf{v}_0) = (x_b - T_{\text{det}}\, v_{x0},\; y_b - T_{\text{det}}\, v_{y0},\; v_{x0},\; v_{y0})$ is the initial condition that maps to $\mathbf{x}_b$ with velocity $\mathbf{v}_0$, and $p(\mathbf{v}_0 \mid \mathbf{x}_b, \eta_{iz})$ is the conditional velocity distribution given final position $\mathbf{x}_b$ (derived below).

3.3 Poisson Observation Model

Conditional on ϕ_i and η_{iz}, pixel counts are independent Poisson:

$$n_{izsb} \sim \text{Poisson}\big(\lambda_{izsb}(\phi_i, \eta_{iz})\big), \qquad s \in \{g, e\},\ b = 1, \ldots, N_b. \tag{10}$$

Independence holds because atoms are independent given $(\phi_i, \eta_{iz}, \Lambda_{iz})$, and the two states are imaged in spatially disjoint pixel sets. The per-shot conditional log-likelihood is

$$\log \mathcal{L}_i(\beta, \phi_i, \eta_{iz}) = \sum_z \sum_{s\in\{g,e\}} \sum_b \Big[n_{izsb} \log \lambda_{izsb}(\phi_i, \eta_{iz}) - \lambda_{izsb}(\phi_i, \eta_{iz}) \Big], \tag{11}$$

where $\log(n!)$ constants are dropped as they are parameter-free.

4 Pixel Statistics via Gauss–Hermite Quadrature

4.1 Conditional Velocity Distribution

We evaluate the velocity expectation in (**??**) analytically for a Gaussian cloud. Under the ballistic map, the joint distribution of (x^f, v_{x0}) factorises as

$$p(x^f, v_{x0} \mid \eta_{iz}) = f_{\text{cloud}}(x^f;\, \eta_{iz})\, p(v_{x0} \mid x^f,\, \eta_{iz}), \tag{12}$$

where the marginal final-position density is

$$f_{\text{cloud}}(x^f;\, \eta) = \mathcal{N}\big(x^f;\ \mu_{x^f},\ \sigma^2_{x^f}\big), \quad \mu_{x^f} = \mu_{x_0} + T_{\text{det}}\, \mu_{v_x}, \quad \sigma^2_{x^f} = \sigma^2_x + T^2_{\text{det}}\, \sigma^2_{v_x}, \tag{13}$$

and the conditional velocity is Gaussian with pixel-dependent moments:

$$p(v_{x0} \mid x^f,\, \eta) = \mathcal{N}\big(v_{x0};\ \tilde{\mu}_{v_x}(x^f),\ \tilde{\sigma}^2_{v_x}\big), \quad \tilde{\mu}_{v_x}(x^f) = \mu_{v_x} + \frac{(x^f - \mu_{x^f})\, \sigma^2_{v_x}\, T_{\text{det}}}{\sigma^2_{x^f}}, \quad \tilde{\sigma}^2_{v_x} = \frac{\sigma^2_{v_x}\, \sigma^2_x}{\sigma^2_{x^f}}. \tag{14}$$

Identical expressions hold for $v_{y0} \mid y^f$. The velocity expectation in (**??**) therefore becomes a 2D integral over two independent conditional Gaussians:

$$\mathbb{E}[p_s(\cdot;\, \Phi)] = \int dv_{x0}\, dv_{y0}\; p(v_{x0} \mid x_b)\; p(v_{y0} \mid y_b)\; p_s\big(\mathbf{q}_0(\mathbf{x}_b, \mathbf{v}_0);\, \Phi\big). \tag{15}$$

4.2 2D Gauss–Hermite Quadrature

The integrand in (**??**) is a smooth function of (v_{x0}, v_{y0}) (the PSMAP is smooth by construction) against a Gaussian measure — the ideal setting for Gauss–Hermite (GH) quadrature. Let $(\xi_k, w_k)_{k=1}^{n_{\text{GH}}}$ be the standard GH nodes and weights. The nodes in physical velocity units at pixel centre $\mathbf{x}_b$ are

$$v_{x0,k}(\mathbf{x}_b) = \tilde{\mu}_{v_x}(x_b) + \sqrt{2}\, \tilde{\sigma}_{v_x}\, \xi_k, \tag{16}$$

and analogously for v_{y0}. The 2D GH approximation to (**??**) is

$$\mathbb{E}[p_s(\cdot;\, \Phi)] \approx \frac{1}{\pi} \sum_{k_x=1}^{n_{\text{GH}}} \sum_{k_y=1}^{n_{\text{GH}}} w_{k_x}\, w_{k_y}\; p_s\big(\mathbf{q}_0(\mathbf{x}_b,\ v_{x0,k_x},\ v_{y0,k_y});\, \Phi\big), \tag{17}$$

converging exponentially in n_{GH} for smooth PSMAP functions. In practice $n_{\text{GH}} = 5$ (25 nodes per pixel per AI) is sufficient.

4.3 Pre-Computed Per-Pixel Statistics

Substituting the cosine structure of (??) into (??) and grouping terms by their dependence on Φ, define three scalars per pixel b, AI z, and output state s:

$$A_{izsb} \;=\; \frac{1}{\pi}\sum_{k_x,k_y} w_{k_x}w_{k_y} \sum_{p:\, s_p=s} \big[a_{0,p}(\mathbf{q}_{0,k})^2 + a_{1,p}(\mathbf{q}_{0,k})^2\big], \tag{18}$$

$$C^c_{izsb} \;=\; \frac{1}{\pi}\sum_{k_x,k_y} w_{k_x}w_{k_y} \sum_{p:\, s_p=s} 2\,\iota_p\, a_{0,p}(\mathbf{q}_{0,k})\, a_{1,p}(\mathbf{q}_{0,k}) \cos\Delta\phi_p(\mathbf{q}_{0,k}), \tag{19}$$

$$C^s_{izsb} \;=\; \frac{1}{\pi}\sum_{k_x,k_y} w_{k_x}w_{k_y} \sum_{p:\, s_p=s} 2\,\iota_p\, a_{0,p}(\mathbf{q}_{0,k})\, a_{1,p}(\mathbf{q}_{0,k}) \sin\Delta\phi_p(\mathbf{q}_{0,k}), \tag{20}$$

where $\mathbf{q}_{0,k} = \mathbf{q}_0(\mathbf{x}_b, v_{x0,k_x}, v_{y0,k_y})$ are the GH quadrature points. These scalars depend on η_{iz} and $\mathbf{x}_b$ but *not* on ϕ_i or β. Combining with (??), the expected pixel count is

$$\boxed{\lambda_{izsb}(\Phi) \;=\; \Lambda_{iz}\,\Delta A\, f_{\text{cloud}}(x_b;\, \eta_{iz})\, f_{\text{cloud}}(y_b;\, \eta_{iz})\, \big[A_{izsb} + C^c_{izsb}\cos\Phi + C^s_{izsb}\sin\Phi\big],} \tag{21}$$

with $\Phi = \Phi_{iz}(\phi_i, \beta)$ from (??). The dependence on ϕ_i and β enters only through a single sinusoid evaluated at the pre-computed $(A_{izsb}, C^c_{izsb}, C^s_{izsb})$. This is the central result enabling efficient GPU evaluation.

5 Handling Unobserved Parameters: Profiling and Marginalisation

The full likelihood involves two kinds of unobserved parameters per shot: the common laser phase ϕ_i, and the cloud parameters η_{iz}. Both must be eliminated from the likelihood before we can optimise over the science parameters β. This section explains the two standard tools for doing so — *marginalisation* and *profiling* — and explains why each is applied to a different type of unobserved parameter here.

5.1 The Problem: Unobserved Parameters in the Likelihood

We want to estimate β from the data. The likelihood we can write down is the *joint* likelihood

$$\log \mathcal{L}_i(\beta, \phi_i, \{\eta_{iz}\}) = \sum_{z,s,b} \Big[n_{izsb} \log \lambda_{izsb}(\phi_i, \eta_{iz}) - \lambda_{izsb}(\phi_i, \eta_{iz})\Big], \tag{22}$$

which also depends on ϕ_i and η_{iz}. These are real parameters of the model — they determine the data-generating process — and they are unknown. We cannot simply ignore them; the likelihood evaluated at the wrong ϕ_i or η_{iz} would give biased or meaningless results.

There are two principled strategies for eliminating unobserved parameters from a likelihood: marginalisation and profiling.

5.2 Marginalisation

What it is. If an unobserved parameter θ has a known prior distribution $p(\theta)$, we can eliminate it by integrating ("summing") the likelihood over all its possible values, weighted by the prior:

$$\mathcal{L}_i^{\text{marg}}(\beta) = \int p(\theta)\; \mathcal{L}_i(\beta, \theta)\; d\theta. \tag{23}$$

Equivalently, $\mathcal{L}_i^{\text{marg}}(\beta) = p(\text{data}_i \mid \beta)$ is the marginal probability of the observed data after integrating out θ. This is the Bayesian-correct thing to do: we are uncertain about θ, and we account for that uncertainty by averaging over it.

When is it necessary? Marginalisation is essential when: (i) the prior on θ is broad or flat (uninformative), and (ii) θ is correlated with the signal β in the likelihood — meaning that if we tried to optimise over θ jointly with β, the optimiser could artificially increase the likelihood by moving θ to absorb the variation that should be attributed to β. In such cases, optimising over θ (rather than marginalising) causes overfitting: θ soaks up the signal.

5.3 Profiling (Concentrated Likelihood)

What it is. For each fixed value of the signal parameters β, we find the value of the nuisance θ that maximises the likelihood:

$$\hat{\theta}(\beta) = \arg\max_{\theta} \log \mathcal{L}(\beta, \theta). \tag{24}$$

The *profile likelihood* (also called the concentrated likelihood) is then

$$\log \mathcal{L}^{\text{prof}}(\beta) = \log \mathcal{L}\big(\beta,\, \hat{\theta}(\beta)\big). \tag{25}$$

Intuitively: at each candidate β, we ask "given this signal, what is the best possible explanation for the data from the nuisance parameters?" and use that as the likelihood for β.

When is it appropriate? Profiling is appropriate when the nuisance θ is *well-constrained by the data* — meaning the likelihood is sharply peaked in θ for any fixed β. When this is the case, the posterior over θ is essentially a delta function at $\hat{\theta}(\beta)$, and profiling $\approx$ marginalising (the two give the same answer to high accuracy). Profiling is simpler: it avoids placing a prior on θ and avoids a potentially high-dimensional integral.

Profiling is *not* appropriate when the prior on θ is important (e.g., improper or flat), because optimising over a flat-prior nuisance that is correlated with β can cause the nuisance to absorb the signal.

5.4 Why Marginalise ϕ_i?

$\phi_i \sim \text{Uniform}(0, 2\pi)$ is drawn anew each shot and is completely unknown. Its prior is flat over a 2π cycle.

If we tried to profile over ϕ_i (jointly optimise with β), the optimiser would set ϕ_i shot-by-shot to match the observed port fractions exactly, regardless of whether those fractions are due to the signal $\delta\phi_i$ or to noise. The per-shot phase ϕ_i has 1 degree of freedom per shot, and the signal $\delta\phi_i(\beta) = A_s \sin(2\pi f t_i) + A_c \cos(2\pi f t_i)$ is only a 2-parameter perturbation spread over all shots — it is far smaller in magnitude than ϕ_i itself. Profiling would absorb the signal entirely into the per-shot ϕ_i estimates, and $\hat{\beta}$ would be biased toward zero.

Marginalisation avoids this. With ϕ_i integrated out, the likelihood for shot i only depends on β through whether different values of β shift the phase-marginalised port fractions in a way consistent with the data. The numerical approximation to the marginal (**??**) is

$$\log \mathcal{L}_i^{\text{marg}}(\beta) = \text{logsumexp}_k\big[\log \mathcal{L}_i(\beta, \phi_k)\big] - \log n_\theta, \tag{26}$$

where $\phi_k = 2\pi k / n_\theta$, $k = 0, \ldots, n_\theta - 1$, is a uniform grid over $[0, 2\pi)$, approximating the integral (**??**) by a Riemann sum in log-space. This is exact as $n_\theta \to \infty$ and accurate for $n_\theta \gtrsim 2\sqrt{N}$ (see Section **??**).

5.5 Why Profile η_{iz}?

Each η_{iz} has 8 components (COM position, COM velocity, and cloud widths in x and y) and varies independently shot to shot. Unlike ϕ_i, there is no reason to assign these a flat prior: the cloud parameters are physically bounded and their shot-to-shot variation, while real, is not so large as to make the likelihood flat in η_{iz}.

More importantly, for each shot i and AI z, the image provides $N_b \times 2$ pixel observations (both states, all pixels), which for a 32×32 grid gives 2048 Poisson measurements to constrain 8 parameters. This is a vastly over-determined system. The likelihood is therefore *very sharply peaked* in η_{iz}: the posterior $p(\eta_{iz} \mid \text{data}_i, \beta)$ is essentially a delta function at the MLE $\hat{\eta}_{iz}(\beta)$ for any fixed β. In this regime, profiling and marginalising are numerically identical — there is no practical difference between them.

We therefore profile: for each candidate β in the outer optimiser, find $\hat{\eta}_{iz}(\beta)$ and evaluate the likelihood there.

Approximate independence from β. There is a further simplification. The cloud parameters η_{iz} control the *spatial* distribution of atoms across pixels. The signal β controls the *port fractions* (the ratio of atoms in state g versus state e). These two effects are largely orthogonal: the sum image (state g + state e) is sensitive to η_{iz} but insensitive to β (the interference term cancels in the sum), while the fringe pattern (state g − state e) is sensitive to β but only weakly sensitive to η_{iz} (through the spatial envelope).

This approximate separability means that $\hat{\eta}_{iz}(\beta)$ barely changes as β varies over its plausible range. We can therefore compute $\hat{\eta}_{iz}$ *once*, at the start of inference (e.g. by optimising over η_{iz} at $\beta = 0$), and treat it as fixed for the outer β optimiser. This turns a nested optimisation problem (inner over η_{iz} at every outer step) into a precomputation followed by a 2D optimisation over β — a large practical gain.

Implementation. The per-shot nuisance MLEs $\hat{\eta}_{iz}$ are computed at construction time (Section **??**). For each shot i and AI z, we run an 8-parameter Nelder–Mead optimiser over the per-shot sum-image Poisson likelihood. This exploits the sum-image separability: the total counts $n^+_{izb} = n_{izgb} + n_{izeb}$ have expected value $\Lambda_{iz}\,\Delta A\, f_{\text{cloud}}(x_b; \eta_{iz})\, f_{\text{cloud}}(y_b; \eta_{iz})\, \bar{p}_{\text{det}}$, where $\bar{p}_{\text{det}}$ is nearly independent of ϕ_i, so the optimisation does not require a phase grid. The resulting $\hat{\eta}_{iz}$ is stored and used to pre-compute the pixel statistics $(A_{izsb}, C^c_{izsb}, C^s_{izsb})$ via GH quadrature, which are then fixed for all subsequent likelihood evaluations.

6 Full Marginal Log-Likelihood

6.1 Per-Shot Marginal Log-Likelihood

Using (**??**) and the profiled $\hat{\eta}_{iz}$, the conditional log-likelihood (**??**) decomposes as

$$\log \mathcal{L}_i(\beta, \phi_k) = \underbrace{\sum_{z,s} \sum_{b:\, n_{izsb}>0} n_{izsb} \log \lambda_{izsb}(\phi_k)}_{\text{sparse sum over occupied pixels}} - \underbrace{\sum_{z,s} \Lambda_{iz} \langle p_s \rangle_{iz,k}}_{\text{global scalar}}, \tag{27}$$

where $\langle p_s \rangle_{iz,k}$ is the mean detection probability at phase grid point ϕ_k. The marginal over ϕ_i is then

$$\log \mathcal{L}_i(\beta) = \text{logsumexp}_k\big[\log \mathcal{L}_i(\beta, \phi_k)\big] - \log n_\theta. \tag{28}$$

6.2 Grid Size for Phase Marginalisation

The integrand in the ϕ_i marginalisation is peaked near the most-likely phase $\hat{\phi}_i$. The peak width scales as $(C\sqrt{N_{\text{atoms}}})^{-1}$, where $C \lesssim 1$ is the contrast. The Riemann sum approximation is accurate when the grid spacing $2\pi/n_\theta$ is small compared to this width:

$$n_\theta \gtrsim 2\sqrt{\bar{N}}, \qquad \bar{N} = \text{mean detected atoms per shot per AI.} \tag{29}$$

In practice n_θ is rounded to the next power of two with a floor of 512.

6.3 Full Log-Likelihood

Since shots are independent given β:

$$\log \mathcal{L}(\beta) = \sum_{i=1}^{N_{\text{shots}}} \log \mathcal{L}_i(\beta). \tag{30}$$

Maximum likelihood estimation solves $\hat{\beta} = \arg\max_\beta \log \mathcal{L}(\beta)$.

7 Atom Number Normalisation

For each AI z and shot i, Λ_{iz} is determined by the requirement that the total predicted detection count matches the observed total:

$$\sum_{b,s} \lambda_{izsb}(\phi_i) = \Lambda_{iz}\, \bar{p}_{\text{det},iz}(\phi_i) \stackrel{!}{=} N_{iz}^+, \qquad N_{iz}^+ = \sum_b (n_{izgb} + n_{izeb}), \tag{31}$$

where $\bar{p}_{\text{det},iz}(\phi_i)$ is the mean detection probability at phase ϕ_i, averaged over the cloud:

$$\bar{p}_{\text{det},iz}(\phi_i) = \sum_s \Delta A \sum_b f_{\text{cloud}}(\mathbf{x}_b;\, \hat{\eta}_{iz})\, \left[A_{izsb} + C^c_{izsb} \cos \Phi_{iz} + C^s_{izsb} \sin \Phi_{iz}\right]. \tag{32}$$

Since $\bar{p}_{\text{det}}$ varies weakly with ϕ_i (the phase modulation of the detection efficiency is second-order), we evaluate it once at $\phi_i = 0$ using the PSMAP at the cloud COM. This gives a fixed pre-computed scalar $\hat{p}_{\text{det},iz}$, and

$$\Lambda_{iz} = \frac{N_{iz}^+}{\hat{p}_{\text{det},iz}}. \tag{33}$$

8 GPU Implementation

8.1 Pre-Computation Phase (Once per Shot per AI)

1. **Cloud nuisance profiling.** Fit $\hat{\eta}_{iz}$ by 8-parameter Nelder–Mead over the per-shot sum-image Poisson likelihood (Section **??**). Starting point: image COM and second moments (fast moment estimators).
2. **Atom number.** Compute Λ_{iz} via (**??**) using N_{iz}^+ and a single PSMAP evaluation at the cloud COM.
3. **GH node construction.** For each pixel b, compute the conditional velocity means and variances from (**??**) at $\mathbf{x}_b$ using $\hat{\eta}_{iz}$, and map the standard GH nodes via (**??**). This produces $N_b \times n_{\text{GH}}^2$ initial-space coordinate tuples $\mathbf{q}_{0,k}(b)$.
4. **Batch PSMAP interpolation.** Evaluate the PSMAP at all $N_b \times n_{\text{GH}}^2$ quadrature points in a single parallelised GPU kernel:

$$\big(a_{0,p}(\mathbf{q}_{0,k}),\ a_{1,p}(\mathbf{q}_{0,k}),\ \Delta\phi_p(\mathbf{q}_{0,k})\big) \ \leftarrow\ \text{PSMAP\_interp}(\mathbf{q}_{0,k}). \tag{34}$$

5. **Pixel statistics.** Accumulate (A_{izsb}, C^c_{izsb}, C^s_{izsb}) per pixel per state via (**??**)–(**??**) using a weighted reduction over the n_{GH}^2 nodes. Result: tensors of shape $(N_b, 2, 3)$ per shot per AI, stored on GPU.

8.2 Per-Evaluation Phase (Every Optimiser Call)

1. **Signal phases.** $\delta\phi_i = A_s \sin(2\pi f\, t_i) + A_c \cos(2\pi f\, t_i)$ for $i = 1, \ldots, N_{\mathrm{shots}}$; vectorised over shots.
2. **Effective phase grid.** For each shot i, AI z, and phase grid point k:

$$\Phi_{iz,k} = \phi_k + \psi_z(\delta\phi_i), \qquad \phi_k = \frac{2\pi k}{n_\theta}, \tag{35}$$

   yielding a tensor of shape $(N_{\mathrm{shots}}, N_z, n_\theta)$.
3. **Expected counts.** For each pixel b, shot i, AI z, state s, phase point k:

$$\lambda_{izsb}(\phi_k) = \Lambda_{iz}\, \Delta A\, f_b^{iz} \left[A_{izsb} + C^c_{izsb} \cos \Phi_{iz,k} + C^s_{izsb} \sin \Phi_{iz,k}\right], \tag{36}$$

   where $f_b^{iz} = f_{\mathrm{cloud}}(x_b; \hat{\eta}_{iz})\, f_{\mathrm{cloud}}(y_b; \hat{\eta}_{iz})$.
4. **Conditional log-likelihood.** Per shot i and phase point k, compute $\log \mathcal{L}_i(\beta, \phi_k)$ via (**??**).
5. **Phase marginalisation.** $\log \mathcal{L}_i(\beta) = \mathrm{logsumexp}_k[\log \mathcal{L}_i(\beta, \phi_k)] - \log n_\theta$.
6. **Shot sum.** $\log \mathcal{L}(\beta) = \sum_i \log \mathcal{L}_i(\beta)$.

8.3 Computational Cost

Step	Phase	Dominant cost	Typical value
Nuisance profiling	Pre-comp.	$N_{\mathrm{shots}} \times N_z$ 8D optimisations	$\sim$1 s total
PSMAP interpolation	Pre-comp.	$N_b\, n_{\mathrm{GH}}^2$ evals/shot/AI	2.5×10^5
Pixel statistics	Pre-comp.	same FLOPs, reduction	<1 ms (GPU)
Expected counts tensor	Per eval	$N_{\mathrm{shots}}\, N_z\, N_b\, n_\theta$ FLOPs	$\sim 10^8$
Sparse log-sum	Per eval	$N_{\mathrm{shots}}\, N_z\, N_b\, n_\theta$	$\sim 10^8$
logsumexp	Per eval	$N_{\mathrm{shots}}\, n_\theta$	negligible

Table 1: Cost breakdown with typical values $N_{\mathrm{shots}} = 100$, $N_z = 2$, $N_b = 1024$ (32×32), $n_\theta = 512$, $n_{\mathrm{GH}} = 5$ (25 nodes/pixel/AI/shot). Pre-computation is performed once; per-evaluation cost is $\sim 5 \times 10^8$ FLOPs, sub-second on a modern GPU.

9 Maximum Likelihood Estimation

9.1 Parameter Space

With f fixed, the only free parameters are $\beta = (A_s, A_c)$. The constraint $A_s \geq 0$ breaks the exact degeneracy $\log \mathcal{L}(A_s, A_c) = \log \mathcal{L}(-A_s, -A_c)$, making the recovered phase $\hat{\varphi} = \arctan 2(\hat{A}_c, \hat{A}_s)$ unique on $(-\pi/2,\, \pi/2]$.

9.2 Multi-Start Nelder–Mead

The marginal log-likelihood $\log \mathcal{L}(A_s, A_c)$ is smooth but can have a local maximum near the null $(0, 0)$ when the signal is weak. We use the following two-stage procedure:

Stage 1 — Coarse scan. Evaluate $\log \mathcal{L}$ at starting points $(A_s^{(m)}, 0)$ for $A_s^{(m)} \in \{0, 0.03, 0.07, 0.10, 0.15, 0.25$ using a coarse phase grid ($n_\theta = 512$), running Nelder–Mead from each to a loose convergence tolerance. The scan identifies the basin containing the global maximum.

Stage 2 — Fine polish. Re-run Nelder–Mead from the Stage-1 best solution using the adaptive n_θ from (**??**) and tight convergence tolerances (`xatol` $= 10^{-8}$, `fatol` $= 10^{-5}$).

10 Validation: Reduction to the Count-Only Likelihood

When the PSMAP is approximately uniform across the cloud (CONFOCAL configuration, no imposed phase ramp, negligible wavefront aberration), the transition probability $p_s(\mathbf{q}_0; \Phi) \approx \bar{p}_s(\Phi)$ is independent of $\mathbf{q}_0$. The GH quadrature then gives pixel-independent statistics:

$$A_{izsb} \to \bar{A}_s, \qquad C^c_{izsb} \to \bar{C}_s \cos \bar{\phi}_s, \qquad C^s_{izsb} \to \bar{C}_s \sin \bar{\phi}_s, \tag{37}$$

and the expected count factorises as

$$\lambda_{izsb}(\Phi) = \Lambda_{iz}\, \Delta A\, f_b \underbrace{\left[\bar{A}_s + \bar{C}_s \cos(\bar{\phi}_s + \Phi)\right]}_{\equiv\, \bar{p}_s(\Phi)}. \tag{38}$$

The Poisson log-likelihood (**??**) then becomes

$$\log \mathcal{L}_i(\beta, \phi_k) = \sum_{z,s} \Big[N_{izs} \log\big(\Lambda_{iz}\, \bar{p}_s(\Phi_{iz,k})\big) - \Lambda_{iz}\, \bar{p}_s(\Phi_{iz,k}) \Big] + \text{const}, \tag{39}$$

where $N_{izs} = \sum_b n_{izsb}$ is the total per-AI per-state count — exactly the count-only Poisson (equivalently Binomial) likelihood used in the existing `LikelihoodEvaluator`.

This equivalence provides the primary validation target: synthetic data generated with negligible wavefront should yield $\hat{\beta}$ values statistically consistent between the position-resolved and count-only pipelines.